Họ và tên: Đinh Tiến Anh - Team Lead & SoC/ Interconnect Architect

BÁO CÁO TUẦN 1

Đồ án thiết kế 1: Khảo sát không gian thiết kế L2 chia sẻ bằng gem5

I. WHAT: Công cụ mã nguồn mở gem5

- Công cụ mã nguồn mở gem5 là là một phần mềm mô phỏng kiến trúc máy tính ở mức hệ thống (System-Level Architecture Simulator) điều khiển bằng sự kiện rời rạc. Nó cho phép các kỹ sư xây dựng một hệ thống vi mạch (SoC) hoàn chỉnh bằng phần mềm — từ lõi CPU, các tầng Cache, mạng liên kết (NoC) cho đến RAM — trước khi chế tạo phần cứng thật.

II. WHY: Tại sao cần sử dụng gem5?

- Tiết kiệm chi phí và giảm thiểu rủi ro: Chế tạo một con chip thử nghiệm (Tape-out) tốn hàng triệu USD. Nếu thiết kế L2 Cache bị nghẽn hoặc NoC quá hẹp, chip sẽ thất bại. gem5 giúp nhóm phát hiện "nút thắt cổ chai" (bottleneck) ngay trên máy tính với chi phí 0 đồng.

- Khả năng quan sát tuyệt đối: Nếu chạy trên board mạch FPGA hoặc chip thật, rất khó để biết tại sao chip chạy chậm. Với gem5, file báo cáo stats.txt cho biết chính xác: Có bao nhiêu gói tin bị kẹt ở NoC? Tỉ lệ trượt (Miss Rate) của L2 là bao nhiêu phần trăm? Độ trễ trung bình là bao nhiêu nano-giây?

- Tính linh hoạt cực cao: Để thay đổi các thông số của L2 như Way, MSHR, hay mở rộng đường truyền (bus width) của NoC từ 64-bit lên 128-bit, trên phần cứng thật phải thiết kế lại toàn bộ mạch. Trên gem5, nhóm chỉ cần thay đổi 1 tham số dòng lệnh là có thể chạy mô phỏng ngay lập tức.

- Cộng đồng sử dụng gem5 tương đối mạnh mẽ ở các diễn đàn nước ngoài, dễ dàng tiếp cận các nguồn tài liệu, hướng dẫn để sử dụng và nghiên cứu các hướng đi mới cho chủ đề.

III. WHEN: Áp dụng khi nào và trong trường hợp cụ thể nào?

- gem5 được sử dụng trong giai đoạn Pre-silicon (nghiên cứu kiến trúc và đánh giá hiệu năng trước khi đội ngũ RTL bắt tay vào viết code Verilog).

- Các trường hợp áp dụng cụ thể trong đồ án "Khảo sát không gian thiết kế L2 chia sẻ":

1. Trường hợp 1: Định cỡ bộ nhớ đệm Cache.

+ Khi nào: Khi nhóm không biết L2 Cache nên để 512KB, 1MB hay 2MB là vừa đủ cho thuật toán xử lý tín hiệu (DSP).

+ Cách làm: Dùng gem5 chạy thử các mức dung lượng khác nhau, trích xuất chỉ số IPC (Lệnh/chu kỳ) để vẽ biểu đồ tìm ra điểm cân bằng tối ưu và phù hợp cho nhu cầu.

2. Trường hợp 2: Kiểm tra băng thông mạng liên kết

+ Khi nào: Khi 4 nhân CVA6 cùng xả dữ liệu, nhóm cần biết liệu kênh truyền (bus width) của NoC có bị quá tải không.

+ Cách làm: Dùng gem5 cấu hình NoC từ 64-bit lên 128-bit. Phân tích số liệu để chứng minh liệu có hiện tượng “bottleneck” (nghẽn cổ chai)

3. Trường hợp 3: Đánh giá sức chịu tải của MSHR.

+ Khi nào: Khi thuật toán DSP truy cập bộ nhớ liên tục.

+ Cách làm: Dùng gem5 để tăng số lượng thanh ghi chờ (MSHR) của L2 từ 16 lên 32, giúp L2 không bị "đóng băng" khi có quá nhiều lệnh trượt Cache cùng lúc.

IV. WHO: Cho ai dùng? Vị trí kỹ sư nào thì nên dùng?

- SoC & Interconnect Architect (như vị trí bản thân đang đảm nhận): thiết kế base CVA6.

- Memory/Cache Architect: Dùng nền tảng gem5 để tinh chỉnh các thông số vi kiến trúc của Cache (Size, Way, …) nhằm tối ưu hóa tỉ lệ Hit/Miss.

- Performance Engineer: Dùng gem5 để phân tích file log (stats.txt), tìm ra điểm nghẽn cổ chai và đề xuất phương án giải quyết cho toàn hệ thống.

V. HOW: Triển khai công việc cụ thể của bản thân

1. Tìm hiểu về các thuật ngữ thường sử dụng:

Thuật ngữ	Diễn giải theo cách hiểu
Design Space Exploration	Quá trình khảo sát các kịch bản cấu hình phần cứng khác nhau => Tìm ra điểm tối ưu, phù hợp nhất giữa 3 thành phần là hiệu năng - diện tích chip và công suất.
IPC (Instructions Per Cycle)	Số lệnh trên mỗi chu kỳ, là số đo hiệu năng quan trọng nhất của CPU
Footprint	Tổng dung lượng dữ liệu mà một chương trình cần sử dụng để chạy
Stride	Khoảng cách giữa 2 lần CPU đọc dữ liệu trong RAM

N-way Set-Associative Cache	Cache được chia thành nhiều Set (tập hợp), mỗi Set chứa N-way (vị trí). Dữ liệu từ RAM có thể nằm ở bất kỳ vị trí nào trong tập hợp được chỉ định
L1 Cache	Chia thành loại I và D: loại I để chứa code, loại D để chứa dữ liệu. Nằm ở sát nhân CPU nhất
L2 Shared Cache	Là một khối lớn mà các nhân CPU dùng chung.
MSHR (Miss Status Holding Register)	Thanh ghi chờ, có 2 loại là Blocking (các thế hệ cũ) và Non-blocking (thế hệ mới). Non-blocking Cache: Khi Miss, L2 ghi vào MSHR, rồi tiếp tục phục vụ các CPU khác. MSHR = 32 => Có thể ghi chờ 32 yêu cầu mà không bị treo hệ thống.
DDR (Double Data Rate)	Chuẩn bộ nhớ RAM (DDR4, DDR5). Truyền dữ liệu ở cả xung lên và xung xuống của clock => x2 tốc độ đọc dữ liệu so với SDRAM.

Bảng 1: Chú thích các thuật ngữ

2. Một số kiến thức dành cho đồ án

- CVA6 (CORE-**V A**pplication-class **6**-stage pipeline processor):

+ **C**ORE-**V:** Dòng vi xử lý mã nguồn mở dựa trên kiến trúc RISC-V

+ **A: A**pplication class (Dòng vi xử lý cấp ứng dụng, có khả năng chạy các hệ điều hành như Linux)

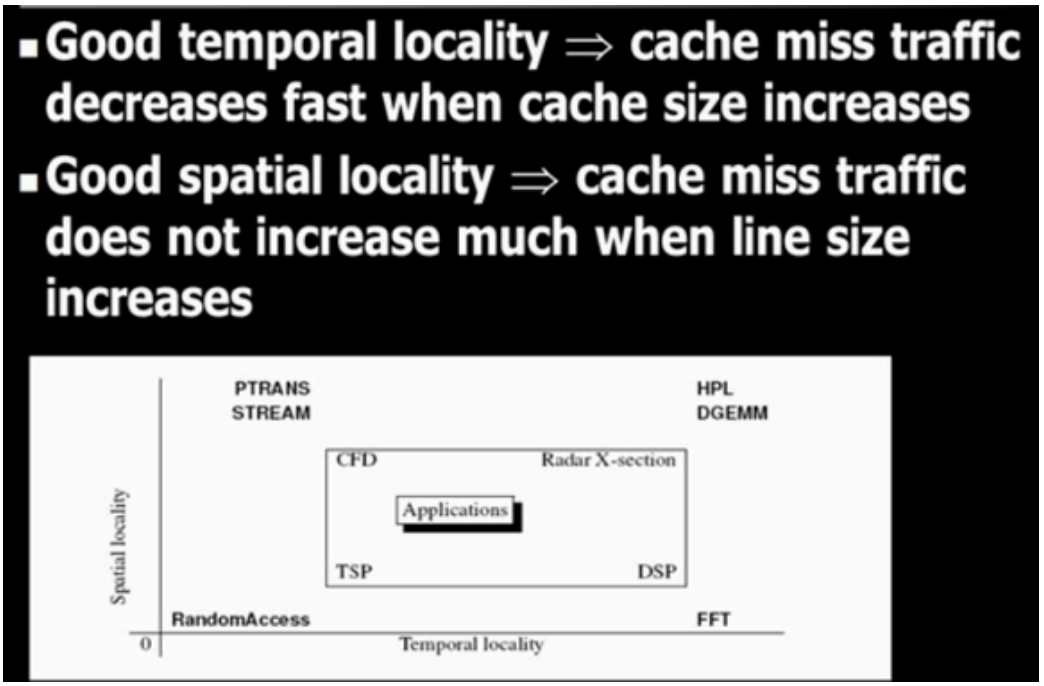
+ **6: 6**-stage execution pipeline (Thiết kế đường ống giải mã và thực thi gồm 6 giai đoạn)

- Nguyên tắc cục bộ: Là nguyên tắc này rất quan trọng trong việc thiết kế hệ thống bộ nhớ, đặc biệt là những hệ thống có nhiều cấp độ bộ nhớ. Có 2 loại cục bộ gồm:

+ Cục bộ thời gian (Temporal locality): đề cập đến xu hướng của một chương trình truy cập vào cùng một vị trí bộ nhớ nhiều lần trong một khoảng thời gian ngắn. Nếu ứng dụng có tính cục bộ thời gian tốt, **chỉ cần tăng dung lượng Cache** lên một chút, tỉ lệ Miss sẽ giảm mạnh.

+ Cục bộ không gian (Spatial locality): đề cập đến xu hướng của

một chương trình truy cập vào các vị trí bộ nhớ gần nhau. Khi tính cục bộ không gian tốt, thiết kế **Cache Line Size lớn (như 64 Bytes hoặc 128 Bytes)** là rất có lợi. Bởi vì khi CPU gọi 1 byte, Cache gọi hết 64 bytes lân cận lên và CPU cũng sẽ dùng các byte lân cận đó, nên đường truyền (traffic) không bị lãng phí. Ngược lại, nếu tính cục bộ không gian kém , gọi 64 bytes lên mà CPU chỉ sử dụng 1 byte thì sẽ làm nghẽn NoC.



Hình 1: Biểu đồ chỉ sự tương quan giữa cục bộ thời gian và cục bộ không gian

- Cơ chế hoạt động của Cache: Hit, Miss và Cache Line

+ **Hit:** CPU tìm thấy dữ liệu trong L1

+ **Miss**: CPU không tìm thấy trong L1=> Xuống L2 tìm. Nếu L2 cũng không có (L2 Miss) => Xuống RAM tìm. Mỗi lần Miss, CPU bị phạt (Miss Penalty) rất nặng về thời gian.

+ **Cache Line:** Như trên đề cập, khi CPU bị Miss và phải xuống RAM lấy 1 byte dữ liệu, nó không lấy 1 byte. Nó sẽ gọi luôn một khối 64 bytes (gọi là Cache Line) mang lên L1. Vì dựa trên nguyên lý cục bộ không gian (Spatial Locality), CPU rất có thể sẽ dùng các byte lân cận ngay sau đó.

3. Thiết kế baseline CVA6

3.1. Các tham số khảo sát không gian thiết kế

- Code Python để khai báo và truyền các biến số như dung lượng của L2 (L2_size), L2_assoc, footprint và stride.

- Hỗ trợ trong việc truyền tham số dễ dàng trên bash/terminal thay vì phải sửa đổi code.

3.2. Định nghĩa phần cứng giả lập (SimObjects)

- L1 Cache: Tối ưu độ trễ (latency) để bám sát theo pipeline của CVA6

- Shared L2 Cache: Số lượng thanh ghi MSHRs để xử lý đồng thời nhiều yêu cầu, tránh hiện tượng nghẽn hệ thống.

- ClusterNoC: Cấu hình băng thông và độ trễ chuyển mạch làm trung gian giữa L1 và L2.

3.3. Khởi tạo cụm đa nhân CVA6

- Thiết lập hệ thống ở chế độ timing để đảm bảo mô phỏng chu kỳ chính xác (cycle-accurate).

- Khởi tạo cụm đa nhân dựa trên mô hình CPU In-order, phản ánh đúng vi kiến trúc thực tế của lõi CVA6.

3.4. Định tuyến và đấu nối mạng (Port Binding)

- Kết nối các khối phần cứng tuân thủ nguyên tắc Master/Slave Port theo luồng dữ liệu sau: Nhân CPU => L1 Cache => ClusterNoC => Shared L2 Cache => System Bus => Memory Controller

3.5. Gán tiến trình làm việc đa nhân

- **Chế độ SE (Syscall Emulation):** Chạy trực tiếp thuật toán mà không cần hệ điều hành. Giúp cô lập phần cứng, loại bỏ nhiễu từ các tác vụ ngầm để đo đạc chính xác hiệu năng thuần túy của L2 và NoC.

- **Chế độ FS (Full System):** Khởi động toàn bộ hệ điều hành (Linux Kernel) và ổ cứng ảo. Dùng để đánh giá mức độ ảnh hưởng của các tiến trình hệ thống, xử lý ngắt (interrupts) và quản lý bộ nhớ lên hiệu năng phần cứng.

- Phân bổ tải trọng xử lý (file binary hoặc disk image) cho từng nhân CPU, thiết lập môi trường đa luồng để tạo áp lực tranh chấp dữ liệu thực tế lên mạng liên kết và bộ nhớ dùng chung.

3.6. Khởi chạy mô phỏng và thu thập số liệu

- Chốt sơ đồ kết nối của toàn hệ thống thông qua đối tượng gốc (Root).

- Sử dụng lệnh instantiate() để gem5 dịch bản thiết kế Python thành các linh kiện phần cứng ảo (C++) và cấp phát bộ nhớ.

- Sử dụng lệnh simulate() để hệ thống chính thức chạy. Đồng hồ mô phỏng sẽ đếm liên tục cho đến khi 4 nhân CPU xử lý xong thuật toán và tự động xuất ra file kết quả (stats.txt).

4. Mở rộng (DDR4 - DDR5 - DDR6; DRAMSim3)

- Tuy đồ án đang hướng tới khảo sát không gian thiết kế L2 trong DDR4 với 4 nhân CPU CVA6 nhưng để hướng tới các công nghệ tiên tiến hiện tại và dự kiến thì bản thân em đánh giá mức độ giải quyết điểm nghẽn cổ chai (Memory Wall) khi cụm 4 nhân CVA6 truy xuất dữ liệu như sau:

Tiêu chí	DDR4	DDR5	DDR6 (dự kiến)
Tốc độ truyền tải	2400-3200 MT/s	4800-8400 MT/s	> 12800 MT/s
Cấu trúc kênh	1 kênh 64-bit /DIMM	2 kênh độc lập 32-bit /DIMM	4 kênh độc lập 16-bit /DIMM
Burst Length	BL8	BL16	BL32
Lợi ích đối với SoC đa nhân	Dễ bị nghẽn khi nhiều nhân cùng truy xuất.	Phục vụ đồng thời 2 luồng dữ liệu, giảm độ trễ hàng đợi.	Tối đa hóa tính song song, giải phóng hoàn toàn L2 MSHR.

Bảng 2: Bảng so sánh các chuẩn bộ nhớ RAM

- Mặc dù gem5 có sẵn bộ điều khiển bộ nhớ (MemCtrl), nhưng để khảo sát chuyên sâu các chuẩn bộ nhớ thế hệ mới (đặc biệt là DDR5 và DDR6), hệ thống cần tích hợp thêm **DRAMSim3** – công cụ mô phỏng bộ nhớ chu kỳ chính xác (cycle-accurate) chuyên biệt:

+ **Về mặt kiến trúc:** gem5 vẫn đảm nhận mô phỏng CPU CVA6, L1, NoC và L2. Tuy nhiên, tại cổng ra của System Bus, thay vì dùng MemCtrl mặc định, hệ thống sẽ gọi giao diện DRAMSim3.

+ **Cơ chế hoạt động:** Khi L2 Cache bị trượt (Miss), gem5 sẽ đóng gói yêu cầu đọc/ghi và gửi sang DRAMSim3. DRAMSim3 tính toán chính xác thời gian hoàn thành dựa trên trạng thái vật lý của RAM, sau đó trả dữ liệu và độ trễ thực tế về lại cho gem5.

+ **Mục tiêu báo cáo:** Sử dụng DRAMSim3 để chứng minh sự đánh đổi (Trade-off) giữa tiêu thụ điện năng và băng thông.

5. Ánh xạ (Mapping) và hiệu chỉnh độ trễ (Tuning Latency) từ CVA6 cho MinorCPU

- Vấn đề: Vì CVA6 có 6 stage pipeline gồm: PC Gen, Fetch, Decode, Issue, Execute, Commit) , còn MinorCPU chỉ có 4 stage pipeline gồm: Fetch1, Fetch2, Decode, Execute nên dẫn đến sự sai lệch lưu lượng truy xuất bộ nhớ, dẫn đến việc thiết kế L2 bị sai.

- Giải pháp:

+ **Can thiệp thẳng vào code C++ của gem5:** Giúp mô phỏng chuẩn xác CVA6, tuy nhiên phải cực kỳ chuyên sâu về engine của gem5, và phải sửa bug, không thể hoàn thành trong thời gian 03 tháng => Không phù hợp

+ **Ánh xạ và tinh chỉnh qua cấu hình Python của MinorCPU:** Sử dụng lại lõi C++ từ cộng đồng gem5 đã được kiểm chứng, thời gian tinh chỉnh chỉ khoảng 01 ngày, kèm theo độ chuẩn xác chỉ đạt ở mức tin tưởng cao (High Fidelity). Đồng thời MinorCPU không đóng vai trò là đối tượng nghiên cứu chính, nên có thể đánh giá là một trade-off phù hợp với nội dung đồ án hiện tại. => Phù hợp

- Cụ thể cách 2, ánh xạ được dựa theo chức năng của từng stage như sau:

+ **[PC Gen + Fetch] (CVA6) => [Fetch1 + Fetch2] (MinorCPU):** Vì cả 2 nhóm tầng này đều làm việc với L1 I-Cache. Tầng PC Gen của CVA6 làm nhiệm vụ tính toán địa chỉ lệnh tiếp theo và gửi yêu cầu (Request) đến L1I. Chức năng này khớp hoàn toàn với tầng Fetch1 của MinorCPU. Tầng Fetch của CVA6 làm nhiệm vụ chờ dữ liệu từ L1I trả về (Response) và đẩy vào hàng đợi lệnh. Chức năng này khớp hoàn toàn với tầng Fetch2 của MinorCPU.

+ **[Decode + Issue] (CVA6) => [Decode] (MinorCPU):** Đây là giai đoạn giải mã lệnh và kiểm tra điều kiện trước khi thực thi. Tầng Decode của CVA6 dịch lệnh RISC-V (32-bit) thành các vi lệnh (micro-ops). Tầng Issue của CVA6 là nơi lệnh phải đứng chờ (Stall) nếu dữ liệu đầu vào chưa sẵn sàng (Scoreboard checking). Trong MinorCPU, tầng Decode đảm nhận việc giải mã, sau đó đẩy lệnh vào một bộ đệm đầu vào của tầng Execute. Bằng cách cộng thêm độ trễ (Forward Delay) vào giai đoạn chuyển giao từ Decode sang Execute trong MinorCPU, ta mô phỏng hoàn hảo khoảng thời gian lệnh phải nằm chờ tại tầng Issue của CVA6.

+ **[Execute + Commit] (CVA6) => [Execute] (MinorCPU):** Là các tầng chịu trách nhiệm tính toán toán học, truy xuất dữ liệu và ghi kết quả. Tầng Execute của CVA6 chứa các bộ ALU (tính toán), FPU (dấu phẩy động) và LSU (đọc/ghi L1 D-Cache). Tầng Commit của CVA6 làm nhiệm vụ ghi kết quả cuối cùng vào tập thanh ghi (Register File) theo đúng thứ tự (In-order). Tầng Execute của MinorCPU bao gồm các Functional Units (đơn vị thực thi) và cơ chế In-order Commit. Do đó, chỉ cần cấu hình độ trễ của các khối ALU/LSU (Functional Unit Latency) trong MinorCPU dài thêm 1 chu kỳ để bao hàm luôn thời gian ghi kết quả của tầng Commit bên CVA6.